

# Hoja informativa: Data pipelines y por qué utilizarlos

## Práctica

```
-- Crear una tabla

CREATE TABLE table_name (primary_key_field_name serial PRIMARY KEY,
                          column_name1 data_type,
                          column_name2 data_type,
                          ...);
```

```
-- Eliminar una tabla

DROP TABLE table_name;
```

```
-- Dar acceso a una tabla

GRANT ALL PRIVILEGES ON TABLE table_name TO username;
```

```
-- Dar permisos para trabajar con claves primarias

GRANT USAGE, SELECT ON SEQUENCE table_name_id_seq TO anthony;
```

```
# Conectar a una base de datos

from sqlalchemy import create_engine

# Establecer parámetros para conectar a una base de datos.
# Los puedes solicitar al administrador de la base de datos.
db_config = {'user': 'my_user',          # nombre de usuario
             'pwd': 'my_user_password', # contraseña
             'host': 'localhost',        # dirección del servidor
             'port': 5432,                # puerto de conexión
             'db': 'db_name'}            # nombre de la base de datos

# Formar string de conexión de la base de datos.
connection_string = 'postgresql://{user}:{pwd}@{host}:{port}/{db}'.format(db_config['user'],
                                                                            db_config['pwd'],
                                                                            db_config['host'],
                                                                            db_config['port'],
                                                                            db_config['db'])

# Conectarse a la base de datos.
engine = create_engine(connection_string)
```

```
# Ejecutar la consulta y almacenar el resultado en un DataFrame.
# SQLAlchemy automáticamente otorgará a las columnas los mismos nombres que en la tabla de la base de datos.
# Solo tendremos que indicar la columna índice con index_col.

query = ''' SELECT column1, column2, column3
            FROM table_name
            '''

data_raw = pd.io.sql.read_sql(query, con = engine, index_col = 'column1')
```

```
# Añadir filas a la tabla
# if_exists = 'replace' - las filas que coinciden se sobrescriben
# if_exists = 'append' - las filas que coinciden se duplican

df.to_sql(name = 'table_name', con = engine, if_exists = 'append', index = False))
```

```
-- Eliminar filas de una tabla utilizando una condición

DELETE FROM table_name WHERE conditions_for_searching_records_to_be_deleted;
```

## Teoría

Los **data pipelines** (pipelines de datos) son programas especiales que se ejecutan de acuerdo a horarios. Recopilan, fusionan, transforman y almacenan datos automáticamente.

La **agregación** es el proceso de agrupar datos y reducir en tamaño.